T.Y. B.TECH CSE (DATA SCIENCE) · TERM I · TAE 2 — WINTER 2026

23UDSPCL3508 / 23UDSPCP3508 · Database Management Systems

Airbnb Property Booking & Reservation Management Database

Live Project Implementation, SQL Execution & Technical Oral Presentation

PREPARED BY

Urjaa Dalvi

PRN: 24ACDS1121073

Roll No: P-60 · Section: TY-DS

EVALUATION

Subject Teacher: Mr. Chinmay Mukim

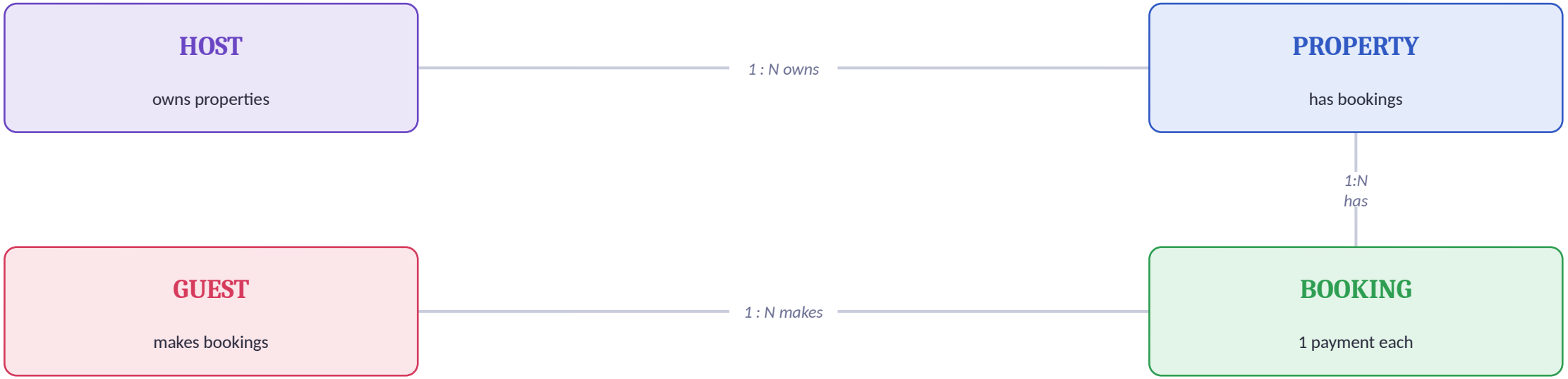
HOD, CSE (CS & DS): Dr. Deepika Ajalkar

Max Marks: 20

SLIDE 1 · 1 MIN

Domain Overview & TAE 1 Recall

AIRBNB PROPERTY BOOKING & RESERVATION MANAGEMENT DATABASE



Business rules carried over from TAE 1

Every property belongs to exactly one host · No two guests can book the same property for overlapping dates · Every booking has exactly one payment · A guest may review a property only after a completed stay

SLIDE 2 · 1 MIN

System Architecture & Implementation Setup

Target Stack

- MySQL 8.0 (per TAE 1 approval)
- Python + Faker + mysql-connector
- MySQL Workbench for the live demo

Schema Topology

- 6 tables: HOST, GUEST, PROPERTY,
- BOOKING, PAYMENT, REVIEW
- CHECK + trigger-enforced business rules

Data Volume

- 40 hosts · 120 guests · 100 properties
- ~350 non-overlapping bookings
- 1:1 payments · reviews on completed stays

Constraint layer: PK/FK, UNIQUE, CHECK, and DEFAULT constraints all live in the schema; overlap and review-eligibility rules live in triggers.

```
e.g.  CHECK (CheckOut_Date > CheckIn_Date)  ·  CHECK (Rating BETWEEN 1 AND 5)  ·  UNIQUE (Booking_ID) on PAYMENT
```

SLIDE 3 · 6 MINS

Live Demonstration Script & Query Benchmarks

1	**Schema Walkthrough**	DESCRIBE each table — highlight the CHECK constraints and the 1:1 Booking–Payment UNIQUE key.
2	**Join Queries**	Inner join (booking+guest+property+host), left outer join (properties never booked), self-join (guests sharing a property).
3	**Procedure & Triggers**	CALL create_booking_with_payment(...); attempt an overlapping booking to show trg_prevent_overlap fire.
4	**Reporting Views**	SELECT * FROM vw_host_performance / vw_property_health — business-ready output.
5	**Performance Benchmark**	EXPLAIN ANALYZE the overlap-check lookup and guest booking history — before vs. after CREATE INDEX.

✦ **Benchmark headline:** A composite index on (Property_ID, Booking_Status, CheckIn_Date, CheckOut_Date) turns the overlap check from a scan into a narrow index range read.

SLIDE 4 · 2 MINS

Technical Q&A Cheat Sheet

ACID Compliance

- Atomicity: booking + payment in one transaction (procedure)
- Consistency: FK/CHECK + triggers reject bad states
- Isolation: InnoDB REPEATABLE READ avoids dirty reads
- Durability: InnoDB redo log survives crashes

Indexing Choices

- Every FK column indexed for joins/lookups
- Composite (Property_ID, Status, dates) for overlap checks
- Composite (Guest_ID, CheckIn_Date) skips a sort

Concurrency Control

- InnoDB row-level locks, not table locks
- Overlap trigger + UNIQUE(Booking_ID) stop double-booking races
- Audit table gives a race-free status history

Defend it in one line: Triggers do the cross-row business rules, InnoDB does the transaction guarantees — indexing is the only thing we chose by hand.